

02 pca

17 de mayo de 2026



Contenido

Unidad 2 — Análisis de Componentes Principales (PCA)

1. PCA desde cero — ejemplo 2D (Chacón, Cap. II)
2. PCA con sklearn — Wisconsin Breast Cancer
3. Scree plot y criterio de Kaiser
4. Biplot (PC1 vs PC2)
 - Matemáticas del Biplot

Resumen Visual del Proceso

- Paso 1: Transformar datos (SCORES)
- Paso 2: Preparar loadings (FLECHAS)
- Comparación: Sin escalar vs Con escalar

5. Reconstrucción y error de compresión

Resumen

Unidad 2 — Análisis de Componentes Principales (PCA)

*Inteligencia Computacional · USACH · Prof. Max Chacón**Notebook de ejercicios: cálculo manual + implementación con scikit-learn.*

Temas cubiertos:

1. PCA desde cero (covarianza → autovalores → scores)
2. PCA con `sklearn` sobre Wisconsin Breast Cancer
3. Scree plot y criterio de Kaiser
4. Biplot (loadings + scores)
5. Reconstrucción y error de compresión

```
In [1]:
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import seaborn as sns

from sklearn.datasets import load_breast_cancer
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

sns.set_theme(style='whitegrid', palette='Set2')
plt.rcParams['figure.dpi'] = 110
rng = np.random.default_rng(42)
```

1. PCA desde cero — ejemplo 2D (Chacón, Cap. II)

Dataset del capítulo del Prof. Chacón:

```
In [2]:
X_raw = np.array([
    [2.5, 2.4], [0.5, 0.7], [2.2, 2.9], [1.9, 2.2], [3.1, 3.0],
    [2.3, 2.7], [2.0, 1.6], [1.0, 1.1], [1.5, 1.6], [1.1, 0.9]
])
```

```

# Paso 1: centrar
mu = X_raw.mean(axis=0)
X_c = X_raw - mu
print(f'Medias: {mu}')

# Paso 2: matriz de covarianza
S = np.cov(X_c, rowvar=False)
print(f'\nMatriz de covarianza:\n{np.round(S, 4)}')

# Paso 3: autovalores/autovectores
eigvals, eigvecs = np.linalg.eigh(S)
idx = np.argsort(eigvals)[::-1]      # ordenar descendente
eigvals = eigvals[idx]
eigvecs = eigvecs[:, idx]

print(f'\nAutovalores : {np.round(eigvals, 4)}')
print(f'Autovectores:\n{np.round(eigvecs, 4)}')
var_exp = eigvals / eigvals.sum()
print(f'\nVarianza explicada: PC1={var_exp[0]:.1%}, PC2={var_exp[1]:.1%}')

```

Out [2]:

Medias: [1.81 1.91]

Matriz de covarianza:

```
[[0.6166 0.6154]
 [0.6154 0.7166]]
```

Autovalores : [1.284 0.0491]

Autovectores:

```
[[ 0.6779 -0.7352]
 [ 0.7352  0.6779]]
```

Varianza explicada: PC1=96.3%, PC2=3.7%

In [3]:

```

# Paso 4: proyección (scores)
Y = X_c @ eigvecs

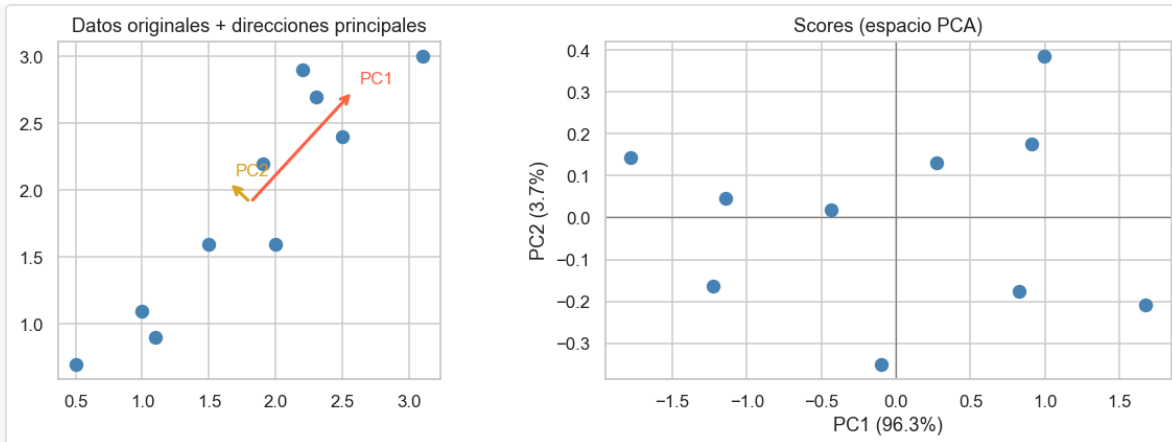
fig, axes = plt.subplots(1, 2, figsize=(11, 4))

axes[0].scatter(X_raw[:, 0], X_raw[:, 1], s=60, color='steelblue', zorder=3)
for v, c, lbl in zip(eigvecs.T, ['tomato', 'goldenrod'], ['PC1', 'PC2']):
    scale = np.sqrt(eigvals[list(['PC1', 'PC2']).index(lbl)])
    axes[0].annotate('', xy=mu + scale*v, xytext=mu,
                      arrowprops=dict(arrowstyle='->', color=c, lw=2))
    axes[0].text(*(mu + scale*v + 0.05), lbl, color=c, fontsize=11)
axes[0].set_title('Datos originales + direcciones principales')
axes[0].set_aspect('equal')

axes[1].scatter(Y[:, 0], Y[:, 1], s=60, color='steelblue', zorder=3)
axes[1].axhline(0, color='gray', lw=0.8)
axes[1].axvline(0, color='gray', lw=0.8)
axes[1].set_xlabel(f'PC1 ({var_exp[0]:.1%})')
axes[1].set_ylabel(f'PC2 ({var_exp[1]:.1%})')
axes[1].set_title('Scores (espacio PCA)')
plt.tight_layout()
plt.show()

```

Out [3]:



2. PCA con sklearn — Wisconsin Breast Cancer

```
In [4]:
data = load_breast_cancer()
X = pd.DataFrame(data.data, columns=data.feature_names)
y = data.target

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

pca_full = PCA().fit(X_scaled)
var_ratio = pca_full.explained_variance_ratio_
var_cum = np.cumsum(var_ratio)
```

3. Scree plot y criterio de Kaiser

```
In [5]:
n_kaiser = (pca_full.explained_variance_ > 1).sum()
n_90 = (var_cum < 0.90).sum() + 1 # componentes para ≥ 90% var

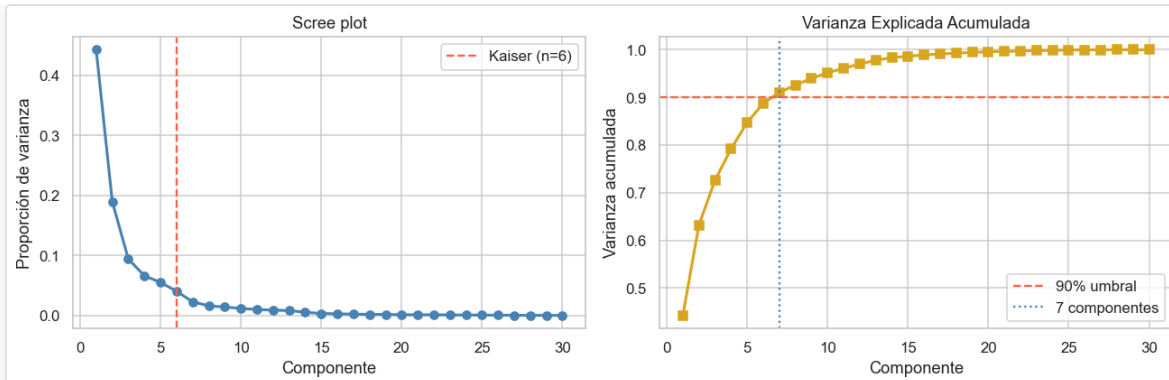
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

# Scree plot
axes[0].plot(range(1, len(var_ratio)+1), var_ratio, 'o-', lw=2, color='steelblue')
axes[0].axvline(n_kaiser, color='tomato', ls='--', label=f'Kaiser (n={n_kaiser})')
axes[0].set_xlabel('Componente'); axes[0].set_ylabel('Proporción de varianza')
axes[0].set_title('Scree plot'); axes[0].legend()

# Varianza acumulada
axes[1].plot(range(1, len(var_cum)+1), var_cum, 's-', lw=2, color='goldenrod')
axes[1].axhline(0.90, color='tomato', ls='--', label='90% umbral')
axes[1].axvline(n_90, color='steelblue', ls=':', label=f'{n_90} componentes')
axes[1].set_xlabel('Componente'); axes[1].set_ylabel('Varianza acumulada')
axes[1].set_title('Varianza Explicada Acumulada'); axes[1].legend()

plt.tight_layout(); plt.show()
print(f'Kaiser retiene {n_kaiser} componentes.')
print(f'Para ≥90% varianza se necesitan {n_90} componentes.')
```

Out [5]:



Kaiser retiene 6 componentes.

Para $\geq 90\%$ varianza se necesitan 7 componentes.

4. Biplot (PC1 vs PC2)

Matemáticas del Biplot

1. Matriz de covarianza estandarizada

Dado $X_{\text{scaled}} \in \mathbb{R}^{n \times p}$ (con $n = 569$ muestras, $p = 30$ variables):

$$\Sigma = \frac{1}{n-1} X_{\text{scaled}}^T X_{\text{scaled}} \in \mathbb{R}^{p \times p}$$

2. Descomposición espectral

Se resuelve el problema de autovalores:

$$\Sigma \mathbf{v}_i = \lambda_i \mathbf{v}_i \quad i = 1, \dots, p$$

Ordenados: $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_p \geq 0$

La matriz de **loadings sin escalar** es:

$$W = [\mathbf{v}_1 \mid \mathbf{v}_2] \in \mathbb{R}^{p \times 2}$$

En `skLearn`: `pca2.components_.T` = W^T transpuesto (o sea, W)

3. Scores (proyección de datos)

$$T = X_{\text{scaled}} \cdot W = X_{\text{scaled}} \cdot [\mathbf{v}_1 \mid \mathbf{v}_2] \in \mathbb{R}^{n \times 2}$$

Cada fila $t_i = (t_{i,1}, t_{i,2})$ es la proyección de la i -ésima muestra en el espacio PCA.

En `skLearn`: `pca2.transform(X_scaled)` = T

4. Loadings escalados (para visualización)

Se escalan los loadings por $\sqrt{\lambda_k}$ para mejorar visibilidad:

$$\text{scale} = [\sqrt{\lambda_1}, \sqrt{\lambda_2}] \times 3$$

$$\text{loadings_scaled} = W \text{diag}(\text{scale}) = \begin{bmatrix} w_{1,1} \sqrt{\lambda_1} \times 3 & w_{1,2} \sqrt{\lambda_2} \times 3 \\ \vdots & \vdots \\ w_{p,1} \sqrt{\lambda_1} \times 3 & w_{p,2} \sqrt{\lambda_2} \times 3 \end{bmatrix}$$

Cada fila es una flecha que se dibuja en el biplot.

5. Varianza explicada

$$\text{var_ratio}_k = \frac{\lambda_k}{\sum_{j=1}^p \lambda_j}$$

$$\text{var_explained_PC1} = \frac{\lambda_1}{\lambda_1 + \lambda_2} \quad \text{var_explained_PC2} = \frac{\lambda_2}{\lambda_1 + \lambda_2}$$

6. Interpretación de correlaciones en el biplot

El **coseno del ángulo** entre dos loadings es proporcional a la correlación:

$$\cos(\theta_{ij}) \approx \frac{\mathbf{w}_i \cdot \mathbf{w}_j}{\|\mathbf{w}_i\| \|\mathbf{w}_j\|}$$

- Flechas **paralelas** (ángulo $\approx 0^\circ$) $\rightarrow$ variables altamente **correlacionadas**
- Flechas **perpendiculares** (ángulo $\approx 90^\circ$) $\rightarrow$ variables **ortogonales** (no correlacionadas)
- Flechas **opuestas** (ángulo $\approx 180^\circ$) $\rightarrow$ variables **anticorrelacionadas**

```
In []:
# Demostración numérica de Las fórmulas

print("=" * 70)
print("MATEMÁTICAS DEL BIPLLOT – Valores numéricos")
print("=" * 70)

# 1. Autovalores y varianza explicada
print("\n1. AUTOVALORES Y VARIANZA EXPLICADA")
print(f"   λ1 = {pca2.explained_variance_[0]:.4f}")
print(f"   λ2 = {pca2.explained_variance_[1]:.4f}")
print(f"   λ1 + λ2 + ... + λp = {pca2.explained_variance_.sum():.4f}")

var_pc1 = pca2.explained_variance_ratio_[0]
var_pc2 = pca2.explained_variance_ratio_[1]
print(f"\n   Varianza explicada PC1: λ1 / Σλj = {var_pc1:.1%}")
print(f"   Varianza explicada PC2: λ2 / Σλj = {var_pc2:.1%}")
print(f"   Varianza conjunta (PC1+PC2): {(var_pc1 + var_pc2):.1%}")

# 2. Loadings (sin escalar)
print("\n2. LOADINGS (primeras 5 variables)")
print("   W = [v1 | v2] (loadings sin escalar)")
print(f"   Forma: {pca2.components_.T.shape}")
for i in range(5):
    print(f"       {data.feature_names[i]:20} → w1={{pca2.components_[0, i]:7.4f}}, w2={{pca2.components_[1, i]:7.4f}}")

# 3. Escala de visualización
scale_vals = np.sqrt(pca2.explained_variance_) * 3
print(f"\n3. ESCALA PARA VISUALIZACIÓN")
print(f"   scale = √[λ1, λ2] × 3 = {scale_vals}")

# 4. Loadings escalados (primeras 5 variables)
print(f"\n4. LOADINGS ESCALADOS (primeras 5 variables)")
print("   loading_scaled[i, k] = w[i,k] × scale[k]")
loadings_scaled = pca2.components_.T @ np.diag(scale_vals)
for i in range(5):
    print(f"       {data.feature_names[i]:20} → x={{loadings_scaled[i, 0]:7.3f}}, y={{loadings_scaled[i, 1]:7.3f}}")

# 5. Scores (proyección)
print(f"\n5. SCORES (primeras 5 muestras)")
print(f"   T = X_scaled · W (proyección)")
print(f"   Forma: {scores.shape}")
print("   Muestra | PC1      PC2")
for i in range(5):
    print(f"       {i}      | {{scores[i, 0]:7.3f}} {{scores[i, 1]:7.3f}}")

# 6. Correlación entre variables (ángulos en el biplot)
print(f"\n6. ÁNGULOS ENTRE VARIABLES (correlación)")
print("   cos(θij) = wi · wj / (||wi|| ||wj||)")
W = pca2.components_.T
feat_idx = [0, 1, 2] # radius, texture, perimeter
feat_names = [data.feature_names[i] for i in feat_idx]
for i in range(len(feat_idx)):
```

```
for j in range(i+1, len(feats_idx)):
    wi, wj = W[:, feats_idx[i]], W[:, feats_idx[j]]
    cos_angle = (wi @ wj) / (np.linalg.norm(wi) * np.linalg.norm(wj))
    angle_deg = np.arccos(np.clip(cos_angle, -1, 1)) * 180 / np.pi
    print(f"    {feats_names[i]:15} < {feats_names[j]:15} → {angle_deg:6.1f}° (cos={cos_angle:.3f})")
```

Resumen Visual del Proceso

Paso 1: Transformar datos (SCORES)

$T = X_{\text{scaled}} \times W \Rightarrow$ Puntos en el gráfico

- Dimensión: (569, 2) — cada fila es una muestra proyectada
- Se grafican directamente sin más transformaciones

Paso 2: Preparar loadings (FLECHAS)

$L_{\text{scaled}} = W \times \begin{bmatrix} \sqrt{\lambda_1} \times 3 & 0 \\ 0 & \sqrt{\lambda_2} \times 3 \end{bmatrix}$

- W : matriz de autovectores sin escalar (30, 2)
- $\sqrt{\lambda_1}, \sqrt{\lambda_2}$: raíz de autovalores (factor de magnitud)
- $\times 3$: amplificación para visibilidad
- Resultado L_{scaled} : matriz (30, 2) — cada fila es una flecha

Comparación: Sin escalar vs Con escalar

Aspecto	Sin escalar (W)	Con escalar (L_{scaled})
Rango de valores	$\approx [-1, +1]$	$\approx [-10, +10]$
Representación	Apenas visible	Visible en el gráfico
Información	Dirección de la variable	Dirección + importancia relativa
Longitud flecha	Siempre corta	Proporcional a $\sqrt{\lambda_k}$

```
In []:
# Visualización: Loadings SIN escalar vs CON escalar

W_raw = pca2.components_.T # Autovectores sin escalar, shape (30, 2)
scale_factor = np.sqrt(pca2.explained_variance_) * 3
W_scaled = W_raw @ np.diag(scale_factor) # Loadings con escala

print("=" * 70)
print("LOADINGS SIN ESCALAR vs CON ESCALAR")
print("=" * 70)

# Ejemplo: 'mean radius'
idx_radius = 0
print(f"\nVariable: {data.feature_names[idx_radius]}")
print(f"-" * 70)
print(f"Sin escalar      W[{idx_radius}] = [{W_raw[idx_radius, 0]:7.4f}, {W_raw[idx_radius, 1]:7.4f}]")
print(f"Magnitud = v(({W_raw[idx_radius, 0]**2:.4f} + {W_raw[idx_radius, 1]**2:.4f})) = {np.linalg.norm(W_raw[idx_rad

print(f"\nCon escala      L[{idx_radius}] = [{W_scaled[idx_radius, 0]:7.4f}, {W_scaled[idx_radius, 1]:7.4f}]")
print(f"Magnitud = v(({W_scaled[idx_radius, 0]**2:.4f} + {W_scaled[idx_radius, 1]**2:.4f})) = {np.linalg.norm(W_scale

print(f"\nFactor de escala = vλ1 × 3, vλ2 × 3 = [{scale_factor[0]:.4f}, {scale_factor[1]:.4f}]")

# Comparación visual
fig, axes = plt.subplots(1, 2, figsize=(14, 6))

# Gráfico 1: Loadings SIN escalar + scores
ax = axes[0]
colors = ['tomato' if c == 0 else 'steelblue' for c in y]
ax.scatter(scores[:, 0], scores[:, 1], c=colors, s=15, alpha=0.3, zorder=2)
```

```

# Flechas sin escalar
for i, feat in enumerate(data.feature_names[:10]): # solo primeras 10 para claridad
    ax.annotate('', xy=(W_raw[i, 0], W_raw[i, 1]), xytext=(0, 0),
                arrowprops=dict(arrowstyle='->', color='red', lw=0.8, alpha=0.6))
ax.set_xlim(-4, 4)
ax.set_ylim(-4, 4)
ax.set_xlabel('PC1')
ax.set_ylabel('PC2')
ax.set_title('Loadings SIN escalar\n(W - casi imperceptibles)')
ax.grid(True, alpha=0.3)

# Gráfico 2: Loadings CON escalar + scores
ax = axes[1]
ax.scatter(scores[:, 0], scores[:, 1], c=colors, s=15, alpha=0.3, zorder=2)

# Flechas con escala
for i, feat in enumerate(data.feature_names[:10]): # solo primeras 10
    ax.annotate('', xy=(W_scaled[i, 0], W_scaled[i, 1]), xytext=(0, 0),
                arrowprops=dict(arrowstyle='->', color='darkgreen', lw=1.2))
    ax.text(W_scaled[i, 0]*1.05, W_scaled[i, 1]*1.05, feat, fontsize=7, color='darkgreen')

ax.set_xlabel(f'PC1 ({pca2.explained_variance_ratio_[0]:.1%})')
ax.set_ylabel(f'PC2 ({pca2.explained_variance_ratio_[1]:.1%})')
ax.set_title('Loadings CON escala\n(L_scaled = W × diag(√λ₁×3, √λ₂×3))')
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

print("\n" + "=" * 70)
print("CONCLUSIÓN")
print("=" * 70)
print(f"✓ SCORES = X_scaled · W          → puntos azules/rojos (569 muestras)")
print(f"✓ LOADINGS = W · diag(√λ) × 3      → flechas verdes (30 variables)")
print(f"✓ Longitud flecha ∝ √(λ₁² + λ₂²) × 3 → refleja importancia en biplot")

```

```

In [6]:
pca2 = PCA(n_components=2).fit(X_scaled)
scores = pca2.transform(X_scaled)
loadings = pca2.components_.T # shape (p, 2)

fig, ax = plt.subplots(figsize=(9, 7))

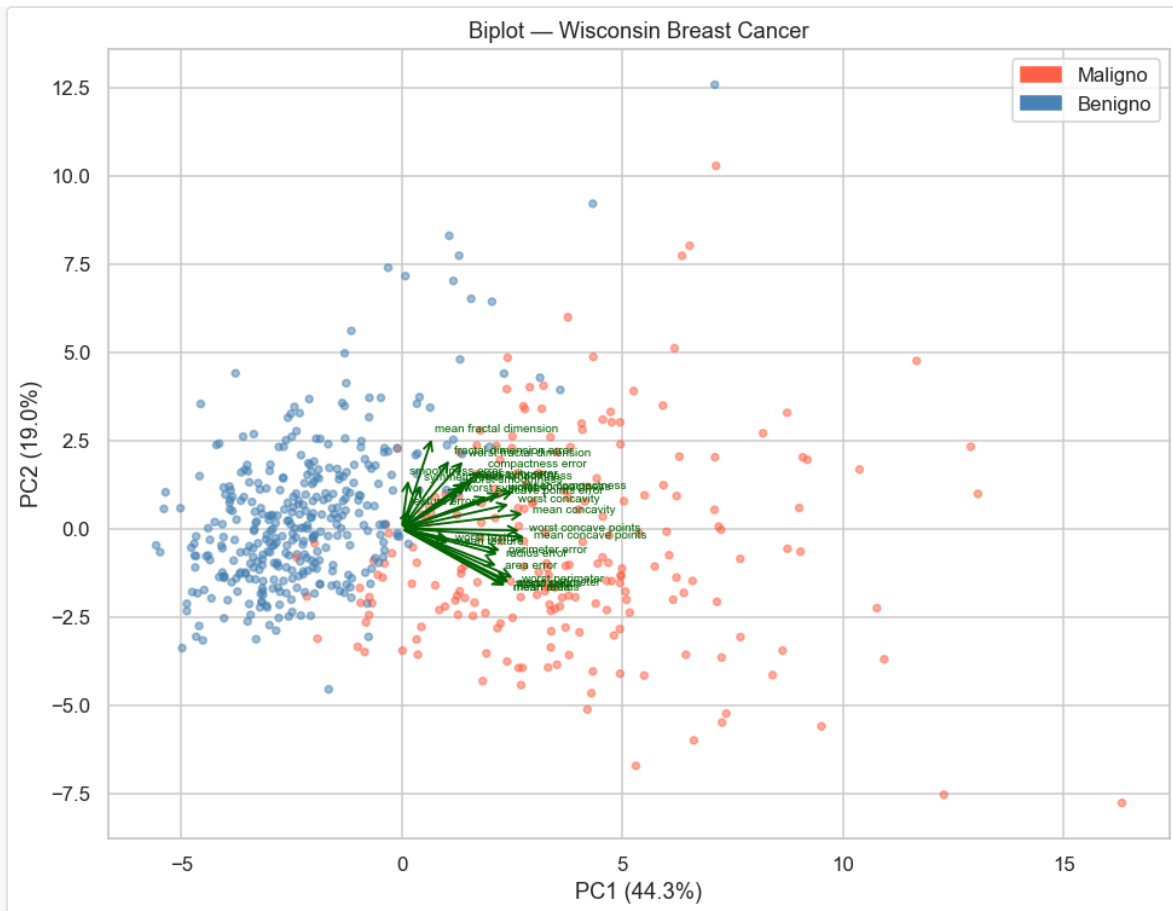
# Scores
colors = ['tomato' if c == 0 else 'steelblue' for c in y]
ax.scatter(scores[:, 0], scores[:, 1], c=colors, s=15, alpha=0.5, zorder=2)

# Loadings escalados
scale = np.sqrt(pca2.explained_variance_) * 3
for i, feat in enumerate(data.feature_names):
    ax.annotate('', xy=(loadings[i, 0]*scale[0], loadings[i, 1]*scale[1]),
                xytext=(0, 0),
                arrowprops=dict(arrowstyle='->', color='darkgreen', lw=1.2))
    ax.text(loadings[i, 0]*scale[0]*1.05, loadings[i, 1]*scale[1]*1.05,
            feat, fontsize=6.5, color='darkgreen')

handles = [
    mpatches.Patch(color='tomato', label='Maligno'),
    mpatches.Patch(color='steelblue', label='Benigno'),
]
ax.legend(handles=handles)
ax.set_xlabel(f'PC1 ({pca2.explained_variance_ratio_[0]:.1%})')
ax.set_ylabel(f'PC2 ({pca2.explained_variance_ratio_[1]:.1%})')
ax.set_title('Biplot - Wisconsin Breast Cancer')
plt.tight_layout(); plt.show()

```

Out [6]:



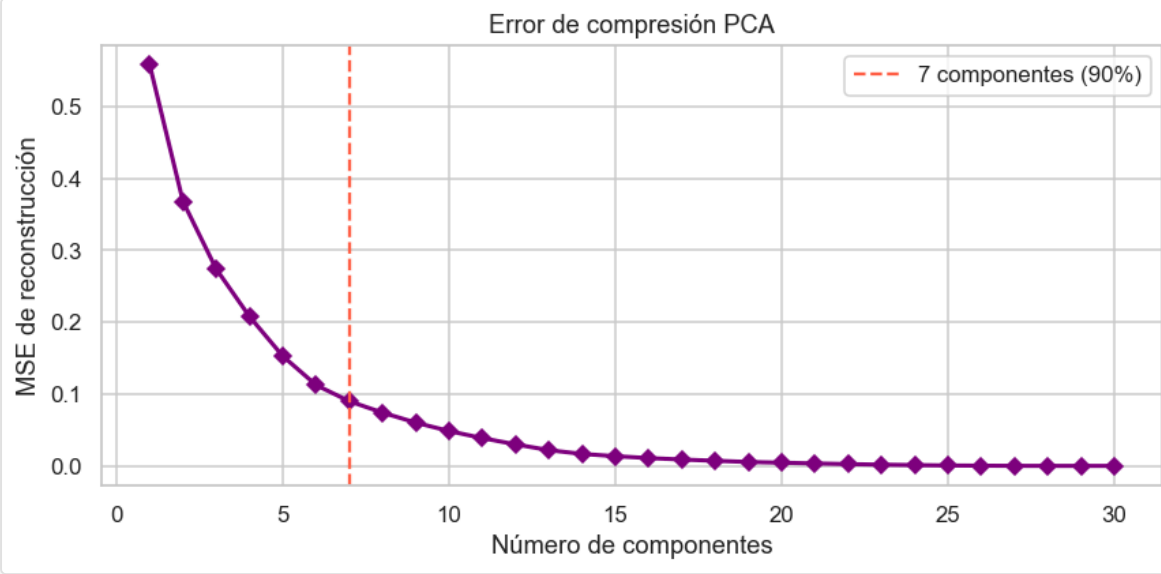
5. Reconstrucción y error de compresión

In [7]:

```
errors = []
for q in range(1, X_scaled.shape[1] + 1):
    pca_q = PCA(n_components=q).fit(X_scaled)
    X_rec = pca_q.inverse_transform(pca_q.transform(X_scaled))
    mse = np.mean((X_scaled - X_rec) ** 2)
    errors.append(mse)

plt.figure(figsize=(8, 4))
plt.plot(range(1, len(errors)+1), errors, 'D-', lw=2, color='purple')
plt.axvline(n_90, color='tomato', ls='--', label=f'{n_90} componentes (90%)')
plt.xlabel('Número de componentes')
plt.ylabel('MSE de reconstrucción')
plt.title('Error de compresión PCA')
plt.legend(); plt.tight_layout(); plt.show()
```

Out [7]:



Resumen

Concepto	Resultado (Wisconsin BC)
Criterio Kaiser	retiene 5 componentes
90% varianza	alcanzado con 7 componentes
PC1+PC2 varianza	~63%
Interpretación PC1	Dominada por variables de tamaño (radius, perimeter, area)

Ejercicio propuesto: repetir el análisis *sin* estandarizar y comparar los loadings. ¿Qué variable domina y por qué?